# VoidCat Worldbuilding & Roleplay Studio

### Product specification — working draft v0.1

The vision: a modular, AI-assisted creative workspace where you build worlds, inhabit them through roleplay, preserve adventures as chapters, and eventually adapt a campaign into a light novel.

The central principle is **consistency**—not rigid uniformity, but fidelity to your artistic intent across lore, characterization, lived scenes, and finished stories.

This is an ambitious target specification, not a commitment to implement everything in the first release. Features below are proposed requirements unless explicitly described as decisions we have already made.

## 1. Product identity and goals

### Product definition

A local-first worldbuilding and roleplay studio that helps creators:

1. Capture and develop fictional worlds.
2. Connect characters, locations, factions, histories, and systems.
3. Test those descriptions through immersive roleplay.
4. Diagnose differences between intended and generated portrayals.
5. Preserve approved adventures as chapters.
6. Refine a campaign into a coherent manuscript.
7. Reuse world material across other creative projects.

### Foundational decisions

- Start independently of production infrastructure.
- Design as a modular part of the broader VoidCat ecosystem.
- Remain useful without image generation, video services, publishing tools, or a particular AI provider.
- Preserve the creator's authority over canon and interpretation.
- Keep world canon, campaign events, character knowledge, and AI suggestions distinct.
- Support both enjoyable RP and deliberate creative testing.

- Treat chapter saving as a core capability.
- Keep campaign-to-light-novel adaptation as the long-term destination.
- Include ensemble RP as an ambitious extension, not a replacement for solo play.

### Non-goals at inception

The first implementation does not need to be:

- An image/video generation marketplace.
- A complete publishing platform.
- A replacement for all existing VoidCat agents.
- A production automation suite.
- An autonomous system authorized to rewrite lore.
- A mechanically complete tabletop RPG engine.

## 2. Core experience

The primary creative loop:

```
Capture an idea
      ↓
Develop and connect the lore
      ↓
Play a scene in that world
      ↓
Annotate what matches or misses the vision
      ↓
Save and approve a chapter
      ↓
Review proposed lore or characterization changes
      ↓
Continue, revise, branch, or adapt
```

Users must also be able to worldbuild without roleplaying, or roleplay without turning every scene into an editorial exercise.

### Primary modes

| Mode | Purpose |
|---|---|
| World Workshop | Develop lore, relationships, systems, and creative direction. |
| Roleplay | Inhabit the setting and interact with its characters. |
| Review | Inspect portrayals, continuity issues, and proposed changes. |
| Chapter Library | Preserve, organize, and revisit campaign history. |
| Manuscript Studio | Adapt selected campaign material into authored fiction. |

Switching modes should preserve the current creative context without silently changing what is canon.

### 3. World and project organization

Proposed hierarchy:

```
Workspace
└── World / Universe
    ├── Lore library
    ├── Continuities
    ├── Reference assets
    └── Creative projects
        ├── RP campaigns
        │   └── Branches → Sessions → Chapters
        ├── Stories and novels
        ├── Songs and concept albums
        └── Other project types
```

#### Requirements

- Support multiple independent worlds.
- Allow several projects to share a world.
- Allow a standalone project to have its own world.
- Support alternate continuities without duplicating every entry.
- Link projects to specific lore revisions.
- Permit explicit migration to newer lore rather than silently changing an ongoing campaign.
- Give entities stable identifiers independent of names and folder locations.

Cross-world references may be supported later, but must not imply shared canon.

### 4. Lore library

#### Entry types

Initial and extensible types:

- Characters.
- Locations.
- Factions and organizations.
- Objects, artifacts, and technology.
- Species and cultures.
- Magic systems and abilities.
- Religions and philosophies.

- Events and historical periods.
- Rules, concepts, themes, and aesthetic guidelines.

### Common entry fields

- Stable ID, name, aliases, and type.
- Description and structured properties.
- Relationships.
- Tags and collections.
- Reference attachments.
- Continuity scope.
- Revision history.
- Source/provenance.
- Author notes and unresolved questions.
- Visibility and knowledge restrictions.

Custom fields and entry templates should be supported.

### Canon and editorial state

Do not collapse all states into one label. Track separately:

| Dimension | Examples |
|---|---|
| Editorial status | Fragment, draft, approved, archived |
| Canon scope | Main continuity, alternate continuity, campaign-only, noncanon |
| Truth framing | Established fact, rumor, belief, disputed account, deliberate unknown |
| Provenance | Author-written, imported, AI-proposed, derived from a scene |

An approved rumor is still a rumor. An archived alternate version should remain recoverable.

### Relationships and timelines

- Directional, typed relationships.
- Time-dependent relationships and changes.
- Objective relationships separate from a character's understanding of them.
- Event ordering even when exact dates are unknown.
- Support for fictional calendars as an advanced feature.
- Graph and timeline views as alternative navigation—not mandatory data-entry methods.

### 5. Fragment inbox

The app must accept incomplete creative material without requiring a finished encyclopedia entry.

Examples include lyrics, dialogue, scene fragments, names, images, questions, and contradictory origin stories.

Users can:

- Capture material quickly.
- Discuss it with AI.
- Link it to existing lore.
- Split it into proposed entries.
- Preserve competing interpretations.
- Leave it intentionally unresolved.

The AI should propose organization rather than force premature decisions.

### 6. AI worldbuilding assistant

#### Capabilities

- Explore implications of an idea.
- Suggest institutions, conflicts, customs, consequences, and relationships.
- Find relevant existing lore.
- Identify possible contradictions.
- Help clarify vague descriptions.
- Generate candidate entries and revisions.
- Explain how a proposal relates to established material.
- Ask targeted questions when ambiguity matters.

#### Change-control contract

```
Discussion → Proposed changes → Creator review → Accepted revision
```

Requirements:

- No silent canon edits.
- Show before/after changes.
- Allow partial acceptance.
- Preserve rejected proposals when useful.
- Attach rationale and supporting material.

- Support undo and revision comparison.
- Identify invention separately from retrieved world facts.

Contradiction detection must allow outcomes such as intentional ambiguity, unreliable narration, historical disagreement, or alternate continuity.

## 7. Roleplay engine

### Session setup

A scene can specify:

- World and continuity.
- Campaign and branch.
- Lore revision baseline.
- User-controlled character or role.
- AI-controlled characters.
- Location and time.
- Opening situation.
- Tone, perspective, and narrative style.
- Boundaries and content preferences.
- Rules and constraints.
- Relevant prior chapters.

### Play styles

- Freeform immersive RP.
- Guided scenario.
- Character interview or dialogue test.
- Relationship test.
- Setting exploration.
- Magic/system stress test.
- Ensemble scene.
- Optional rules-assisted play.

The system should not take control of the user's character beyond permissions the user explicitly chooses.

### Narrator/director role

An optional narrator can manage environment, pacing, transitions, and consequences.

Narrator access to information does not grant characters that knowledge.

### Mechanical rules

Narrative instructions and enforceable rules must be distinguishable.

For example:

- Soft guidance: magic should feel costly.
- Explicit rule: a character has three charges remaining.
- Validated action: an ability cannot be used with zero charges.

Advanced rule packs can enforce selected mechanics. The app must not imply that all prose-defined rules are mechanically validated.

## 8. Ensemble roleplay

Carry forward the Communicator-inspired idea as a separate capability.

### Proposed features

- Multiple distinct AI-controlled participants.
- Per-character identity, knowledge, goals, and memory.
- Manual or automatic turn selection.
- Directed dialogue and group interaction.
- Sequential, moderated, and open-ended modes.
- Configurable limits on autonomous conversation.
- Pause, interrupt, and redirect controls.

### Boundaries

- Fictional characters remain separate from operational VoidCat agents.
- No inherited infrastructure permissions or private agent memories.
- Shared scene state does not mean shared private knowledge.
- Participation orchestration does not require a separate model call for every character; implementation can vary.

### 9. Artistic-vision feedback and diagnostics

Users can annotate any response or passage:

- Matches my vision.
- Out of character.
- Wrong motivation.
- Wrong voice.
- Lore contradiction.
- Knowledge leak.
- Rule violation.
- Tone mismatch.
- Interesting discovery.
- Intentional exception.

Freeform explanations are essential.

Example:

> "The action is right, but she does it out of obligation—not affection."

#### Diagnostic workflow

The app should help investigate whether a mismatch arose from:

1. Missing or ambiguous lore.
2. Relevant lore not reaching the model.
3. Conflicting instructions.
4. Model interpretation or generation failure.
5. Incorrect session state.
6. A genuine creative decision the author wants to reconsider.

It must not automatically assume the lore is wrong.

#### Replay tools

- Branch from a selected point.
- Replay using revised descriptions.
- Compare outputs side by side.
- Preserve model/settings/context metadata.
- Maintain reusable test scenarios.

Different outputs are evidence for the creator to assess, not objective scores of artistic quality.

## 10. Chapters and campaign memory

Chapter saving is a first-class workflow, not merely chat export.

### Closing a chapter

1. Choose a meaningful boundary.
2. Preserve the source messages.
3. Generate a proposed narrative summary.
4. Review, edit, or regenerate it.
5. Approve and save.
6. Continue within the same campaign.

Saving a chapter must not require permanently ending the campaign.

### Chapter record

- Title and sequence.
- Campaign, branch, and continuity.
- Source message range or references.
- Participating characters and locations.
- In-world time when known.
- Narrative summary.
- Important events.
- Relationship changes.
- State changes and consequences.
- Open threads.
- Author notes.
- Approval and revision history.

### Separate outputs

Keep these distinct:

- Transcript: the preserved conversation.
- Chapter summary: approved episodic memory.
- Prose adaptation: an edited literary rendering.

Editing a summary must not rewrite the transcript. A summary is not automatically a finished novel chapter.

### Continuity

- Use approved chapters as a primary source for subsequent campaign memory.
- Retrieve original passages when finer detail matters.
- Distinguish approved memory from provisional live-session state.
- Correct errors without deleting history.
- Keep memories branch-scoped.
- Respect which characters witnessed or learned an event.

## 11. Canon promotion and branching

An RP discovery can be:

- Kept only in the current scene.
- Accepted as a campaign event.
- Proposed as world lore.
- Added to an alternate continuity.
- Rejected as an unwanted generation.

Promoting an event to world canon should present conflicts and affected material for review.

### Branching requirements

- Fork at a message, scene boundary, or chapter.
- Record ancestry.
- Preserve the lore baseline.
- Isolate subsequent memories and consequences.
- Compare alternate outcomes.
- Cherry-pick selected discoveries through explicit review.

No automatic merging of incompatible timelines.

## 12. Campaign-to-light-novel pipeline

This is the long-term destination.

The objective is not “summarize the chat more elegantly.” It is an editorial workspace for transforming interactive material into a deliberate narrative.

### Adaptation stages

| Stage | Proposed function |
| --- | --- |
| Selection | Choose campaign branch, chapters, and scenes. |
| Story analysis | Identify arcs, conflicts, turning points, and unresolved threads. |

| Stage | Proposed function |
|---|---|
| Outline | Organize volumes, acts, chapters, and viewpoint structure. |
| Adaptation | Convert selected material into prose. |
| Revision | Refine characterization, pacing, voice, and continuity. |
| Manuscript assembly | Organize approved prose into a coherent book. |
| Export | Produce portable manuscript deliverables. |

### Editorial capabilities

- Choose perspective, tense, voice, and target audience.
- Define volume/chapter length goals.
- Remove RP repetition and conversational scaffolding.
- Combine, split, reorder, or omit scenes.
- Expand implied transitions with author approval.
- Track newly invented material.
- Preserve important dialogue selectively.
- Separate adaptation continuity from campaign continuity.
- Maintain a style sheet and character-voice guidance.
- Track foreshadowing and payoff.
- Review scenes individually rather than accepting a whole-book rewrite.

### Traceability

```
Manuscript passage
        ↓
Adapted scene
        ↓
Approved chapter
        ↓
Original RP messages
```

The relationship may be many-to-many. A manuscript scene can combine several RP scenes, and some prose may be newly authored.

Changing the manuscript must not silently rewrite the campaign.

## 13. Context and memory architecture

Proposed context layers:

1. Author intent and project direction.
2. Selected world/continuity lore.

3. Character identity and knowledge.
4. Campaign and branch state.
5. Approved chapter memory.
6. Recent scene context.
7. Current user input.

### Requirements

- Assemble context within a configurable token budget.
- Expose what was supplied to a generation.
- Warn when important material is omitted or truncated.
- Preserve source references.
- Support scoped retrieval.
- Avoid treating all retrieved text as authoritative instructions.
- Track generated summaries and their approval state.
- Make context inspection available in Workshop/Review mode without cluttering immersive play.

## 14. Modular architecture and integrations

Proposed logical modules:

```
World Core
├── Lore and relationships
├── Continuity and revisions
└── Canon review

Narrative Runtime
├── Context assembly
├── RP orchestration
├── Session state
└── Character knowledge

Campaign Library
├── Transcripts
├── Chapters
└── Branches

Manuscript Studio
├── Outlining
├── Adaptation
└── Editorial revisions

Adapters
├── Model providers
├── Imports/exports
```

```
├── Agent interfaces
└── Future production tools
```

These are module boundaries, not a requirement for microservices.

### Integration contract

- Versioned interfaces.
- Stable IDs.
- Scoped read/write permissions.
- External agents submit proposals rather than unrestricted canon edits.
- Optional integration events for approved revisions and chapters.
- No requirement to connect production systems to use the app.

Future consumers could request approved character briefs, location references, scene context, or manuscript excerpts.

## 15. Model, cost, privacy, and ownership controls

### Model flexibility

- Local and remote adapters.
- Different models for RP, summaries, retrieval assistance, and editing.
- Provider-neutral stored world data.
- Explicit fallback behavior.
- Per-job model and settings history.

### Cost transparency

- Estimates before paid jobs when available.
- Actual usage recording when reported.
- Session, campaign, and monthly budgets.
- Autonomous-turn and retry limits.
- Cancellation controls.
- No silent paid fallback.
- No claim that local compute has zero operating cost.

### Data controls

- Local editing and browsing without an AI connection.
- Provider keys stored outside browser-delivered code.
- Disclosure of material sent to remote providers.

- Workspace isolation.
- Backup and restore.
- Deletion and retention controls.
- No training or public sharing by default.

### 16. Import, export, and interoperability

Proposed targets:

- Markdown and structured JSON for worlds.
- Transcript and chapter exports.
- Reference attachments with a portable manifest.
- Character-card and lorebook compatibility where feasible.
- Human-reviewed import mapping.
- Duplicate/conflict handling.
- Manuscript text and document formats.
- EPUB as a later publishing-oriented export.

Compatibility limitations must be reported. Unsupported fields should not disappear silently.

### 17. Commercial direction

Potential positioning:

> Build your world. Live its stories. Keep the chapters. Shape the novel.

The commercial hypothesis is an integrated authoring loop—not a claim that lore-aware RP itself is new.

Possible offerings to evaluate later:

- Self-hosted personal edition.
- Hosted convenience edition.
- Team collaboration.
- Optional managed inference.
- Creator templates and scenario packs.

Commercial requirements:

- Portable user data.
- Transparent billing.
- Clear ownership and contributor permissions.
- No artificial dependency on proprietary generation credits.
- No publication or distribution without explicit approval.

Licensing and reuse of existing code require a separate review before a commercial implementation.

## 18. Delivery roadmap

| Phase | Scope | Exit condition |
|---|---|---|
| 0 — Foundation | World entries, fragments, revisions, basic export | A world can be developed without RP or cloud services. |
| 1 — Playable loop | Solo RP, scoped context, annotations, chapter saving | A user can play, approve a chapter, and continue coherently. |
| 2 — Consistency tools | Diagnostics, lore proposals, context inspection, replay | A mismatch can be investigated and revised without overwriting history. |
| 3 — Campaign depth | Branching, knowledge boundaries, timelines, ensemble RP | Alternate campaigns remain meaningfully isolated. |
| 4 — Authoring studio | Outline, scene adaptation, manuscript revisions | A campaign arc can become a traceable prose draft. |
| 5 — Productization | Collaboration, packaging, onboarding, commercial operations | Others can reliably use the system without developer intervention. |

## 19. Critical acceptance scenarios

The eventual implementation should demonstrate that:

1. A character can be renamed without breaking references.
2. An AI proposal cannot become canon without the configured approval.
3. A secret does not automatically become every character's knowledge.
4. Saving a chapter preserves the source transcript.
5. Editing a summary leaves the original conversation intact.
6. A campaign can continue from approved chapter memory.
7. A branch does not inherit events created after its fork point.
8. A revised lore description can be tested against an earlier scene.
9. A generated portrayal can be traced to its supplied context.
10. A manuscript adaptation can diverge without rewriting the campaign.
11. External services can be disconnected while local world editing remains usable.
12. World data can be exported and restored with stable references.

## 20. Decisions intentionally left open

- Product name and visual identity.
- Implementation stack and storage engines.
- Desktop packaging versus browser-first delivery.

- Exact compatibility targets.
- Which rules need deterministic enforcement.
- Default ensemble orchestration.
- Collaboration and permission model.
- Hosting, pricing, and licensing.
- Extent of code reuse versus interface-level integration with Communicator.

Those decisions should follow the product boundaries rather than define them prematurely.

The first milestone remains small enough to recognize: build one world fragment into a playable scene, preserve the experience as an approved chapter, and continue without losing the character's identity or the world's rules. The full specification carries that same thread all the way to a light novel.

⁂

1. https://openart.ai/suite/characters-and-worlds